

可変長引数の定義 — printf の側に立つ

今日のゴール

`int sum(int n, ...)` のような可変長引数の関数を、自分で定義できるようにする。
引数レジスタをフレームに退避し、添字で読み出す仕組みを作る。

1 この回の位置づけ

- 言語機能発展シリーズ（選択制）。前提はコマ 11 (`printf` の呼び出し)
- `language_spec.md` が「ユーザー定義の可変長引数関数（定義）は実装コスト大」として除外した機能の回収
- R2（自前 `printf`）が `printf1(fmt, iarg, sarg)` という不格好な形を強いられた、その理由を解消する回
- `mycc.py` も `scaffold/parser.py` も書き換えない。ラッパー `varcc.py`（完成品）がパーサとコード生成の両方に差し込む
- 編集するのは `variadic.py` の 4 つの関数

2 呼ぶのはできるが、定義はできない

コマ 11 から `printf("x=%d\n", 42)` を呼んできた。呼び出し側は簡単である。引数を `a0`, `a1`, ... に並べて `call` するだけで、個数が可変であることを気にする必要がない。

難しいのは呼ばれる側である。

```
int sum(int n, ...) {  
    /* 引数がいくつ来たのか？ どこにあるのか？ */  
}
```

いまのコンパイラでこれを書くと、構文は通るが `...` は捨てられる。パーサが `...` を読み飛ばしているの、コード生成器は「ただの 1 引数関数」だと思い込んだままになる。

3 問題 1: レジスタはすぐ消える

関数に入った直後、引数は `a0～a7` に入っている。しかし、これらはすぐ上書きされる。

```
sum:
    addi sp, sp, -32
    ...
    li a0, 0          ← この時点で第 1 引数は消えた
```

コマ 8 で「引数はプロローグでスタックに保存する」と学んだが、それは名前のついた引数だけだった。名前のない引数（... の分）は保存されないまま捨てられている。

そこで、関数の先頭で `a0～a7` を 8 本まとめてフレームに書き出す。この領域を **register save area** と呼ぶ。

```
sd a0, -24(s0)      ← save area の先頭
sd a1, -32(s0)
sd a2, -40(s0)
...
sd a7, -80(s0)
```

「まだ何もしていない」タイミングでしか正しい値が読めないので、本体の最初の命令として行う必要がある。

4 問題 2: 添字で読めなければ意味がない

保存できても、`__arg(3)` のように実行時に決まる添字で読めなければ、ループで引数を回せない。

そこで save area を連続した領域として確保する。`alloc_local` を 8 回続けて呼ぶと、8 バイトずつ下がる連続領域になる。

```
s0 - 24    a0 の値      ← 先頭 (base)
s0 - 32    a1 の値
s0 - 40    a2 の値
...
s0 - 80    a7 の値
```

すると i 番目の引数のアドレスは、掛け算と引き算で求まる。

```
アドレス = base - i * 8
```

コマ 10 の `a[i] = base + i * サイズ` と同じ計算で、向きが下向きなだけである。

5 `__arg(i)` —組み込み関数という手

本物の C は `va_start` / `va_arg` / `va_end` というマクロで これを行う。しかし Core プロファイルには関数形式マクロがないので、この回ではコンパイラが特別扱いする組み込み関数 `__arg(i)` を用意する。

```
int __arg(int i);           /* 宣言だけ書く。実体は要らない */

int sum(int n, ...) {
    int total;
    int i;
    total = 0;
    for (i = 1; i <= n; i = i + 1) {
        total = total + __arg(i);    /* i 番目の引数 */
    }
    return total;
}
```

`__arg(i)` は関数呼び出しの形をしているが、`call` は生成されない。コード生成器が「この名前の呼び出しを見たら、`save area` からの読み出しに置き換える」という特別扱いをする。

添字はすべての引数を 0 から数える。`sum(3, 10, 20, 30)` なら `__arg(0)` が $n (=3)$ 、`__arg(1) ~ __arg(3)` が可変長の分になる。

補足

組み込み関数 (intrinsic) という考え方。「関数の顔をしているが、コンパイラが直接コードに置き換えるもの」を組み込み関数 (intrinsic) という。`__builtin_va_arg` (gcc) や `__builtin_popcount` などが実物である。ライブラリでは書けない処理 (レジスタを直接触る、特殊命令を出す) を言語の枠内で書けるようにするための逃げ道であり、実務でもよく使われる。

6 呼び出し側は何も変えなくてよい

面白いことに、呼び出し側のコード生成は 1 行も変えない。`sum(3, 10, 20, 30)` は、コマ 8 で作った通常の呼び出し（引数を `a0`～`a3` に置いて `call`）のままで正しく動く。

可変長引数が「呼ぶ側は普通、呼ばれる側だけ特別」という設計になっているのは、まさにこのためである。

7 実装

Step	実装対象	内容
1	<code>reserve_save_area</code>	8 個のスロットを連続して確保する
2	<code>gen_save_registers</code>	本体の先頭で <code>a0</code> ～ <code>a7</code> を書き出す
3	<code>is_arg_builtin /</code> <code>gen_arg_access</code>	<code>__arg(i)</code> を読み出しに変換する

パーサ側（... があつた関数名を覚えておく）はラッパーが担当する。`scaffold` のパーサが... を捨ててしまうので、トークン列を見て... の有無を調べ直すという方法をとっている。

```
python3 varcc.py tests/sum.c | grep "sd a7"      # save area ができているか
python3 varcc.py tests/sum.c | grep -c "call __arg"  # 0 のはず（呼び出しは生成されない）
```

8 テスト

```
python3 check.py      # Step ごとの単体テスト（未実装は SKIP）
python3 golden.py     # 可変長関数のテスト + fixed15
```

テスト	確認内容
sum.c	引数の個数を変えて合計する（0 個の場合も）
max.c	ループで全引数を走査して最大値を取る
named.c	名前つき引数も <code>__arg</code> で読める / 通常の関数が壊れていない

9 発展課題

1. **R2 の `printf` を作り直す:** `printf1(fmt, iarg, sarg)` をやめ、本物と同じ `int my_printf(char *fmt, ...)` にする。書式指定子を読むたびに `__arg(i)` の `i` を進める
2. **9 個以上の引数:** いまは `a0～a7` の 8 個しか保存していない。9 個目以降は呼び出し側がスタックに積む規約なので、`save area` の「続き」としてそこも読めるようにする
3. **`va_list` 相当:** `__arg(i)` は「何番目か」を呼ぶ側が管理する。代わりに「次の引数を返す」形（`va_arg` 相当）にするには、どこに状態を持てばよいか
4. **型の問題:** いまは全部 8 バイトとして読んでいる。`char *` を渡したときはたまたま動くが、`double` があったらどうなるか（RV64 の浮動小数点は別のレジスタ `fa0～fa7` を使う）
5. **無駄を減らす:** 可変長でない関数では `save area` を作っていないことを、生成コードで確認する。作ってしまうと全関数のフレームが 64 バイト増える

補足

コラム: なぜ可変長引数は「危険」と言われるのか。 `printf("%d %d", 1)` と書いても、コンパイラは止めてくれない（引数が足りないので、2 つ目にはゴミが読まれる）。呼ばれる側は「いくつ来たか」を知る方法がなく、書式文字列や番兵（NULL 終端）など別の手段で教えてもらえないからである。いま作った `__arg(i)` も、`n` を超えた `i` を渡せば `save area` の外を読む。C++ の可変長テンプレートや Rust のマクロが、型と個数を静的に検査する方向へ進化したのは、この弱点が理由である。